Architecture and Technical Specification

CAutomation / CCore / Pipeline Management

Document status: Generation-readiness draft contract for the Pipeline Management reference project. Only this ATS and the matching SRS are manually authored and approved at module level. This iteration hardens table names, field expectations, API shapes, frontend screens, repository targets, validation envelopes, and traceability requirements for deterministic generation.

1. Architecture Boundary and Traceability Rule

This ATS defines how the approved SRS is implemented. It is an engineering contract for database schema, entity relationships, APIs, DTOs, services, UI behavior, validation implementation, testing, reporting, and AI Engine boundaries. It updates the earlier model by separating reusable Task from PipelineTask association records.

- No ATS table, endpoint, field, or service may introduce new business scope.
- Generated implementation must fail when a required SRS-to-ATS mapping is missing.
- The repository structure is not changed by this specification iteration.
- The module path remains cautomation/projects/pipeline_management/input/modules/pipeline_management.
- Only the module SRS and ATS are changed in this iteration.

2. Layered Architecture Contract

Layer	Responsibility
Frontend	Renders hierarchy management, reusable task catalog, pipeline task management, execution status, validation errors, and reports.
API	Exposes deterministic endpoints scoped by hierarchy identifiers and pipeline context identifiers.
Service	Owns lifecycle transitions, validation orchestration, transaction boundaries, and business rule enforcement.
Repository	Performs persistence operations without adding business behavior.
Database	Enforces referential integrity, many-to-many task reuse, uniqueness, status values, timestamps, and audit fields.
Reporting	Persists and exposes generated execution and validation reports.
AI Engine	Consumes approved SRS and ATS only; produces generated artifacts and reports.

3. Database Schema Contract

The schema must implement Project, Product, Module, Pipeline, Task, PipelineTask, PipelineExecution, PipelineTaskExecution, and ExecutionReport as distinct concepts. Pipeline is created in selected Project/Product/Module context. Task is reusable. PipelineTask is the association table that implements the many-to-many relationship and stores per-pipeline usage details.

Table	Purpose	Required fields
project_type	Lookup table for project classification.	id, code, name, description, is_active, created_at, updated_at
project_status	Lookup table for project lifecycle status.	id, code, name, description, is_active, sort_order
project_target	Lookup table for project delivery target.	id, code, name, description, is_active
project	Top-level project record.	id, project_type_id, project_status_id, project_target_id, code, name, description, owner, created_at, updated_at, archived_at
product_type	Lookup table for product classification.	id, code, name, description, is_active
product_status	Lookup table for product lifecycle status.	id, code, name, description, is_active, sort_order
product_target	Lookup table for product delivery target.	id, code, name, description, is_active
product	Product under a project.	id, project_id, product_type_id, product_status_id, product_target_id, code, name, description, owner, created_at, updated_at, archived_at
module_type	Lookup table for module classification.	id, code, name, description, is_active
module_status	Lookup table for module lifecycle status.	id, code, name, description, is_active, sort_order
module_target	Lookup table for module delivery target.	id, code, name, description, is_active
module	Module under a product.	id, product_id, module_type_id, module_status_id, module_target_id, code, name, description, owner, created_at, updated_at, archived_at
pipeline	Versioned pipeline definition created inside selected Project/Product/Module context.	id, project_id, product_id, module_id, code, name, description, version, status, created_at, updated_at, archived_at
task	Reusable task catalog item.	id, task_key, name, description, task_type, status, default_expected_input, default_expected_output, created_at, updated_at, archived_at
pipeline_task	Association between one pipeline and one reusable task.	id, pipeline_id, task_id, sequence_order, is_required, configuration_json, retry_policy_json, failure_behavior, status,

		created_at, updated_at, archived_at
pipeline_execution	Execution instance for a pipeline version.	id, pipeline_id, pipeline_version, execution_key, status, started_at, completed_at, requested_by, summary_message
pipeline_task_execution	Execution instance for one PipelineTask.	id, pipeline_execution_id, pipeline_task_id, task_id, task_key, sequence_order, status, started_at, completed_at, message, error_code
execution_report	Report linked to pipeline or task execution.	id, pipeline_execution_id, pipeline_task_execution_id, report_type, status, title, content_path, summary, created_at, reviewed_at

4. Relationship and Constraint Contract

Relationship	Constraint	Implementation rule
project.project_type_id	FK to project_type.id	Required before activation.
product.project_id	FK to project.id	Cascade delete is not allowed; archive parent instead.
module.product_id	FK to product.id	Module must remain inside one product.
pipeline.project_id	FK to project.id	Pipeline context starts at selected project.
pipeline.product_id	FK to product.id	Product must belong to pipeline.project_id.
pipeline.module_id	FK to module.id	Module must belong to pipeline.product_id.
pipeline_task.pipeline_id	FK to pipeline.id	Association belongs to one pipeline.
pipeline_task.task_id	FK to task.id	Association references one reusable task.
pipeline_execution.pipeline_id	FK to pipeline.id	Execution must reference executed pipeline.
pipeline_task_execution.pipeline_execution_id	FK to pipeline_execution.id	Task execution belongs to one execution.
pipeline_task_execution.pipeline_task_id	FK to pipeline_task.id	Task execution references the pipeline-specific task placement.
pipeline_task_execution.task_id	FK to task.id or snapshot field	Used for traceability to reusable task identity.
execution_report.pipeline_execution_id	FK to pipeline_execution.id	Report must be traceable to an execution.
execution_report.pipeline_task_execution_id	Nullable FK to pipeline_task_execution.id	Required only for task-level reports.

5. Uniqueness and Status Rules

Scope	Constraint
project	code unique globally or within configured tenant/project-family boundary.
product	project_id plus code unique.
module	product_id plus code unique.
pipeline	project_id plus product_id plus module_id plus code plus version unique.
task	task_key unique in reusable task catalog.
pipeline_task	pipeline_id plus task_id unique unless duplicate usage is explicitly enabled later by SRS change.
pipeline_task	pipeline_id plus sequence_order unique.
lookup tables	code unique per lookup table.
execution tables	execution_key unique.
status columns	Limited to approved lifecycle values or approved lookup values.

6. API Contract

Method	Endpoint	Purpose
GET	/api/projects	List projects with metadata filters.
POST	/api/projects	Create project.
GET	/api/projects/{projectId}	Get project detail.
PUT	/api/projects/{projectId}	Update project.
POST	/api/projects/{projectId}/archive	Archive project.
GET	/api/projects/{projectId}/products	List products under project.
POST	/api/projects/{projectId}/products	Create product under project.
GET	/api/products/{productId}/modules	List modules under product.
POST	/api/products/{productId}/modules	Create module under product.
GET	/api/tasks	List reusable tasks for selection.
POST	/api/tasks	Create reusable task.
PUT	/api/tasks/{taskId}	Update reusable task.
GET	/api/projects/{projectId}/products/{productId}/modules/{moduleId}/pipelines	List pipelines in selected context.
POST	/api/projects/{projectId}/products/{productId}/modules/{moduleId}/pipelines	Create pipeline in selected context.
PUT	/api/pipelines/{pipelineId}	Update pipeline definition.
POST	/api/pipelines/{pipelineId}/validate	Validate pipeline definition.
POST	/api/pipelines/{pipelineId}/activate	Activate valid pipeline.
GET	/api/pipelines/{pipelineId}/tasks	List PipelineTask associations ordered by sequence.

POST	/api/pipelines/{pipelineId}/tasks	Add reusable Task to Pipeline as PipelineTask.
PUT	/api/pipeline-tasks/{pipelineTaskId}	Update PipelineTask order/configuration/status.
DELETE	/api/pipeline-tasks/{pipelineTaskId}	Archive or remove PipelineTask from draft pipeline according to lifecycle rules.
POST	/api/pipelines/{pipelineId}/executions	Create pipeline execution.
GET	/api/pipeline-executions/{executionId}	Get execution detail and task statuses.
GET	/api/pipeline-executions/{executionId}/reports	List execution reports.

7. DTO and Error Contract

DTOs must preserve field paths and rule identifiers. Validation errors must be machine-readable and stable enough for generated tests. A validation response contains success, errors, warnings, entity_type, entity_id, rule_id, field_path, and message.

DTO	Required responsibility
ProjectDto/ProductDto/ModuleDto	Expose id, parent id, code, name, description, type/status/target ids and labels, audit fields, and archived flag.
PipelineDto	Expose id, project id, product id, module id, code, name, description, version, status, pipeline task count, and validation summary.
TaskDto	Expose id, task_key, name, description, task_type, status, default expected input, and default expected output.
PipelineTaskDto	Expose id, pipeline id, task id, task_key, sequence_order, is_required, configuration, retry policy, failure behavior, and status.
PipelineExecutionDto	Expose id, pipeline id, version, execution_key, status, timestamps, requested_by, and summary.
PipelineTaskExecutionDto	Expose pipeline_task identity, reusable task identity, task_key, sequence_order, status, timestamps, message, and error code.
ExecutionReportDto	Expose report id, report type, title, status, summary, content path, timestamps, and execution references.

8. Service Contract

Service	Responsibilities
ProjectService	Create, update, list, get, validate, archive, restore projects and enforce metadata references.
ProductService	Manage products inside project boundary and enforce parent integrity.
ModuleService	Manage modules inside product boundary and enforce parent integrity.
TaskService	Manage reusable task catalog and prevent unsafe edits to task identities used by immutable executions.
PipelineService	Manage contextual pipeline definitions, activation, archive, restore, and validation orchestration.
PipelineTaskService	Manage association of reusable Tasks to Pipelines and enforce sequence, configuration, and task_key rules.
PipelineExecutionService	Create execution snapshots, transition statuses, and expose execution detail.
ReportingService	Create and retrieve reports linked to execution or task execution.
MetadataService	Read and manage lookup values where permitted.

9. Validation Implementation Contract

SRS rule	Implementation requirement
SRS-VAL-001	Required-field validators execute before persistence.
SRS-VAL-002	Uniqueness checks execute at service level and are backed by database unique constraints.
SRS-VAL-003	Lifecycle transition validator rejects invalid transitions before update.
SRS-VAL-004	Metadata reference validator rejects inactive lookup values during activation.
SRS-VAL-005	Pipeline context validator verifies project/product/module consistency and active state.
SRS-VAL-006	Pipeline activation validator requires one or more active PipelineTask records.
SRS-VAL-007	Task order validator rejects duplicate or non-positive sequence_order values.
SRS-VAL-008	PipelineTask validator rejects inactive or missing reusable Task references during activation.
SRS-VAL-009	Execution immutability validator prevents definition-style edits to execution records.
SRS-VAL-010	Report validator requires valid execution reference.

10. Frontend Contract

- The UI must show the current hierarchy path: CAutomation / CCore / Pipeline Management.
- Pipeline creation must require selected Project, Product, and Module context.
- The hierarchy browser must allow users to move from Project to Product to Module to Pipeline.

- The task catalog view must allow users to create, search, view, update, archive, and restore reusable Tasks where permitted.
- The pipeline task editor must allow users to add reusable Tasks to a Pipeline and configure sequence_order, required flag, configuration, retry policy, and failure behavior.
- Forms must render server-side validation errors using rule_id and field_path.
- Execution detail must show pipeline status, task statuses, timestamps, reusable task keys, PipelineTask sequence order, and linked reports.
- Archive and restore actions must be explicit and visually distinct from normal update actions.

11. Reporting Implementation

Report type	Minimum content
Planning report	Generated agile artifacts, traceability to SRS requirements, and detected specification gaps.
Validation report	Validation result, rule identifiers, affected entities, and blocking/non-blocking classification.
Execution summary report	Pipeline execution status, started/completed timestamps, PipelineTask status summary, and failure summary.
Task report	Task-level status, reusable task key, PipelineTask configuration reference, input/output references, messages, and error codes.
Apply/verification report	Artifacts applied, verification result, and rollback or follow-up notes when applicable.

12. Testing Contract

Test type	Required coverage
Database tests	Foreign keys, many-to-many Pipeline/Task association, uniqueness constraints, required fields, and lifecycle-safe archive behavior.
Service tests	Validation rules, lifecycle transitions, transactional behavior, context validation, task reuse, and parent-boundary enforcement.
API tests	CRUD endpoints, task catalog endpoints, PipelineTask endpoints, validation endpoint, execution endpoints, report endpoints, error contract, and authorization hooks.
Frontend tests	Form rendering, hierarchy navigation, reusable task selection, PipelineTask ordering, validation display, archive/restore confirmation, execution and report views.
Generation tests	Generated artifacts trace to SRS/ATS and do not invent unsupported entities or endpoints.
Regression tests	Existing cautomation files outside the two specification documents remain unchanged.

13. AI Engine Boundary Rules

- Context Engineering compiles approved SRS and ATS into deterministic context packages.
- Planning primarily consumes SRS functional content and produces agile artifacts.
- Generation consumes both SRS and ATS to produce database, backend, frontend, API, validation, test, and report artifacts.
- Validation checks generated artifacts against SRS and ATS contracts.
- Apply modifies only approved target files after validation passes.
- Verification confirms implementation behavior and report completeness.
- When information is missing, the AI Engine produces a specification gap instead of inventing requirements.

14. Definition of Done

- All tables, APIs, DTOs, services, frontend views, validation rules, tests, and reports trace to the approved SRS.
- The Project to Product to Module context is implemented consistently for pipeline creation.
- Pipeline and Task are implemented as separate entities connected through PipelineTask many-to-many association records.
- Metadata lookup tables exist for Project, Product, and Module type/status/target dimensions.
- Execution and reporting records are immutable where required and traceable to pipeline, PipelineTask, and reusable Task definitions.
- Generated tests cover positive flows, validation failures, lifecycle transitions, task reuse, execution snapshots, and report generation.
- No files outside the agreed generation/apply scope are modified.

15. Detailed Field Matrix

Table	Field	Purpose	Rule
project	id	Primary key	Required, immutable
project	code	Stable code	Required, unique in scope
project	name	Display name	Required

project	description	Long text	Optional
project	created_at	Timestamp	System-managed
project	updated_at	Timestamp	System-managed
project	archived_at	Timestamp	Nullable
project	project_type_id	Metadata reference	Required before activation
project	project_status_id	Status reference	Required
project	project_target_id	Target reference	Required
project	owner	Owner	Required where configured
product	id	Primary key	Required, immutable
product	code	Stable code	Required, unique in scope
product	name	Display name	Required
product	description	Long text	Optional
product	created_at	Timestamp	System-managed
product	updated_at	Timestamp	System-managed
product	archived_at	Timestamp	Nullable
product	project_id	Parent project	Required foreign key
product	product_type_id	Metadata reference	Required before activation
product	product_status_id	Status reference	Required
product	product_target_id	Target reference	Required
product	owner	Owner	Required where configured
module	id	Primary key	Required, immutable
module	code	Stable code	Required, unique in scope
module	name	Display name	Required
module	description	Long text	Optional
module	created_at	Timestamp	System-managed
module	updated_at	Timestamp	System-managed
module	archived_at	Timestamp	Nullable
module	product_id	Parent product	Required foreign key
module	module_type_id	Metadata reference	Required before activation
module	module_status_id	Status reference	Required
module	module_target_id	Target reference	Required
module	owner	Owner	Required where configured
pipeline	id	Primary key	Required, immutable
pipeline	project_id	Selected project context	Required FK
pipeline	product_id	Selected product context	Required FK and must belong to project
pipeline	module_id	Selected module context	Required FK and must belong to product
pipeline	code	Pipeline code	Required, unique with context and version
pipeline	name	Pipeline name	Required
pipeline	description	Description	Optional
pipeline	version	Pipeline version	Required
pipeline	status	Lifecycle status	Required
pipeline	created_at	Timestamp	System-managed
pipeline	updated_at	Timestamp	System-managed
pipeline	archived_at	Timestamp	Nullable
task	id	Primary key	Required, immutable
task	task_key	Reusable task key	Required, unique
task	name	Task name	Required
task	description	Task description	Optional
task	task_type	Task classifier	Required
task	status	Lifecycle status	Required
task	default_expected_input	Default input contract	Optional
task	default_expected_output	Default output contract	Optional
task	created_at	Timestamp	System-managed
task	updated_at	Timestamp	System-managed
task	archived_at	Timestamp	Nullable
pipeline_task	id	Primary key	Required, immutable
pipeline_task	pipeline_id	Pipeline reference	Required FK
pipeline_task	task_id	Reusable task reference	Required FK
pipeline_task	sequence_order	Execution order	Required positive integer unique per pipeline
pipeline_task	is_required	Required flag	Required boolean
pipeline_task	configuration_json	Usage-specific configuration	Nullable JSON
pipeline_task	retry_policy_json	Usage-specific retry policy	Nullable JSON
pipeline_task	failure_behavior	Failure handling behavior	Required default
pipeline_task	status	Lifecycle status	Required
pipeline_task	created_at	Timestamp	System-managed
pipeline_task	updated_at	Timestamp	System-managed
pipeline_task	archived_at	Timestamp	Nullable
pipeline_execution	id	Primary key	Required, immutable
pipeline_execution	pipeline_id	Executed pipeline	Required FK
pipeline_execution	pipeline_version	Executed version	Required snapshot
pipeline_execution	execution_key	Execution key	Required, unique
pipeline_execution	status	Execution status	Required

pipeline_execution	started_at	Start timestamp	Nullable until start
pipeline_execution	completed_at	End timestamp	Nullable until terminal
pipeline_execution	requested_by	Requester	Required where configured
pipeline_execution	summary_message	Execution summary	Optional
pipeline_task_execution	id	Primary key	Required, immutable
pipeline_task_execution	pipeline_execution_id	Parent execution	Required FK
pipeline_task_execution	pipeline_task_id	PipelineTask reference	Required FK or immutable snapshot reference
pipeline_task_execution	task_id	Reusable task reference	Required traceability field
pipeline_task_execution	task_key	Reusable task key snapshot	Required
pipeline_task_execution	sequence_order	Execution order snapshot	Required
pipeline_task_execution	status	Task execution status	Required
pipeline_task_execution	started_at	Start timestamp	Nullable until start
pipeline_task_execution	completed_at	End timestamp	Nullable until terminal
pipeline_task_execution	message	Status message	Optional
pipeline_task_execution	error_code	Error code	Optional
execution_report	id	Primary key	Required, immutable
execution_report	pipeline_execution_id	Execution reference	Required FK
execution_report	pipeline_task_execution_id	Task execution reference	Nullable
execution_report	report_type	Report classifier	Required
execution_report	status	Report status	Required
execution_report	title	Report title	Required
execution_report	content_path	Report content path	Required when report body stored externally
execution_report	summary	Report summary	Optional
execution_report	created_at	Creation timestamp	System-managed
execution_report	reviewed_at	Review timestamp	Nullable

16. Seed Data Contract

Lookup area	Minimum seed values	Rule
Project Status	Draft, Active, Suspended, Archived	Codes must be stable and uppercase or snake_case according to project convention.
Product Status	Draft, Active, Suspended, Archived	Used before activation and in list filters.
Module Status	Draft, Active, Suspended, Archived	Used before pipeline creation.
Pipeline Status	Draft, Active, Deprecated, Archived	Can be enum or lookup table, but generated tests must assert allowed values.
Task Status	Draft, Active, Disabled, Archived	Disabled tasks cannot be selected into newly activated pipelines.
PipelineTask Status	Active, Disabled, Archived	Disabled associations are excluded from new executions.
Execution Status	Pending, Running, Passed, Failed, Cancelled	Terminal statuses are Passed, Failed, and Cancelled.
Task Execution Status	Pending, Running, Passed, Failed, Skipped	Skipped is terminal for task execution.
Report Status	Generated, Reviewed, Archived	Reviewed is a user-facing review state.
Targets	Web Application, Mobile Application, Backend Service, AI Engine, Documentation	Seed values may expand later through metadata management.
Types	Platform, Product, Module, Automation, Integration, Reporting	Seed values must not force future projects to change schema.

17. Transaction and Concurrency Rules

Operation	Transaction rule	Concurrency rule
Create/update hierarchy record	Single transaction per record.	Reject update if optimistic version or updated_at has changed where supported.
Create reusable task	Single transaction for task record.	Reject duplicate task_key before commit and back with unique constraint.
Create pipeline with tasks	Pipeline and PipelineTask associations must commit together when submitted as one change.	Duplicate task order or invalid task reference must be detected before commit and backed by constraints.
Validate pipeline	Read-only transaction where possible.	Validation reads a consistent snapshot of pipeline, task catalog, and PipelineTask associations.
Activate pipeline	Validation and status change must occur in one transaction.	Concurrent edits after validation must force revalidation.
Start execution	Pipeline execution and task execution rows must commit together.	Execution snapshots the pipeline version, PipelineTask order, and reusable task keys used at start time.

Complete task execution	Task execution status update and report creation must be atomic when report is produced.	Terminal task execution cannot be overwritten by stale updates.
Archive record	Archive flag/timestamp update must not physically delete dependent records.	Archived parent cannot receive new active children.

18. Endpoint Response and Status Code Contract

Condition	HTTP status	Response requirement
Successful list or detail read	200	Return DTO or paged DTO list with trace_id when supported.
Successful create	201	Return created DTO and location or identifier.
Successful update/archive/restore/activation	200	Return updated DTO or operation result.
Validation failure	400 or 422	Return validation envelope with rule_id and field_path for each error.
Not authorized	403	Return safe error without leaking unavailable entity details.
Not found inside allowed scope	404	Return not-found error using stable error code.
Conflict or stale update	409	Return conflict code and reload guidance.
Unexpected failure	500	Return trace_id and generic message; internal details go to logs.

19. Repository and Query Rules

Query category	Required rule
Hierarchy list queries	Filter by parent identifier and exclude archived records by default.
Pipeline context queries	Filter by selected project_id, product_id, and module_id, and verify hierarchy consistency.
Detail queries	Verify parent boundary where route hierarchy provides parent identifiers.
Metadata queries	Return active values by default; admin views may include archived values.
Task catalog queries	Search task_key, name, description, and task_type; exclude archived tasks by default.
PipelineTask queries	Order by sequence_order ascending and include reusable task labels.
Execution detail queries	Return pipeline execution plus task executions ordered by sequence_order.
Report queries	Filter by pipeline_execution_id and optionally pipeline_task_execution_id.
Search queries	Search code, name, and description where supported; never cross unauthorized boundaries.

20. Implementation Traceability Matrix

SRS area	ATS implementation artifacts	Generated evidence
Hierarchy model	Database tables, APIs, DTOs, services, frontend hierarchy browser.	Schema tests, API tests, frontend navigation tests.
Metadata model	Lookup tables, metadata API, metadata service, form selectors.	Seed tests and validation tests for inactive lookup values.
Reusable task catalog	task table, TaskService, task APIs, task selectors, validation.	Task CRUD tests and task reuse tests.
Pipeline definition	pipeline and pipeline_task tables, services, forms, validation endpoint.	Create/update/activation tests and task ordering tests.
Task many-to-many model	pipeline_task association table and constraints.	Tests proving one Task can appear in multiple Pipelines.
Execution lifecycle	pipeline_execution and pipeline_task_execution tables and services.	Execution creation, status transition, and immutable execution tests.
Reporting	execution_report table, reporting service, report endpoints and views.	Report creation and retrieval tests.
AI boundary	Context, planning, generation, validation, apply, verification contracts.	Generation validation report showing no invented requirements.

End of ATS.

21. Generation Target Path Contract

Generated artifacts for this module must be staged before apply. The generator must not write directly into live implementation folders. The following paths define deterministic target intent for generation and validation; this specification iteration does not create those files manually.

Artifact group	Staging path pattern	Final target intent
Database schema	.cautomation_workspace/staging/{execution_id}/database/	cautomation/generated/pipeline_management/database/

Backend models	.cautomation_workspace/staging/{execution_id}/backend/models/	cautomation/generated/pipeline_management/backend/models/
Backend services	.cautomation_workspace/staging/{execution_id}/backend/services/	cautomation/generated/pipeline_management/backend/services/
Backend APIs	.cautomation_workspace/staging/{execution_id}/backend/api/	cautomation/generated/pipeline_management/backend/api/
Frontend screens	.cautomation_workspace/staging/{execution_id}/frontend/screens/	cautomation/generated/pipeline_management/frontend/screens/
Validation	.cautomation_workspace/staging/{execution_id}/validation/	cautomation/generated/pipeline_management/validation/
Tests	.cautomation_workspace/staging/{execution_id}/tests/	cautomation/generated/pipeline_management/tests/
Reports	.cautomation_workspace/staging/{execution_id}/reports/	cautomation/generated/pipeline_management/reports/

22. Exact API Payload Shape Contract

Generated APIs must use stable JSON property names. Additional implementation-only metadata may be added only if it is traceable and does not change the business contract.

Operation	Request shape	Response shape
Create Project	{code, name, description, project_type_id, project_status_id, project_target_id}	ProjectDto
Create Product	{project_id, code, name, description, product_type_id, product_status_id, product_target_id}	ProductDto
Create Module	{product_id, code, name, description, module_type_id, module_status_id, module_target_id}	ModuleDto
Create Pipeline	{project_id, product_id, module_id, code, name, description, version, status}	PipelineDto
Create Task	{task_key, name, description, task_type, status, default_input_contract, default_output_contract}	TaskDto
Add Task to Pipeline	{pipeline_id, task_id, sequence_order, is_required, configuration_json, retry_policy_json, failure_behavior, status}	PipelineTaskDto
Start Pipeline Execution	{pipeline_id, requested_by, execution_parameters_json}	PipelineExecutionDto with ordered PipelineTaskExecutionDto list
Create Report	{pipeline_execution_id, pipeline_task_execution_id, report_type, title, summary, content_path}	ExecutionReportDto

23. Validation Envelope Contract

All validation failures must use a deterministic envelope so generated frontend and tests can rely on stable field paths and rule identifiers.

Field	Required meaning	Example
success	Boolean operation result.	false
trace_id	Correlation identifier for logs and reports.	exec-20260704-001
entity_type	Entity being validated.	PipelineTask
entity_id	Identifier when available.	pt-123
errors	Array of blocking validation errors.	[{rule_id, field_path, message}]
warnings	Array of non-blocking findings.	[{rule_id, field_path, message}]
rule_id	Stable SRS/ATS validation rule id.	SRS-VAL-007
field_path	Stable dotted or bracketed path.	pipeline_tasks[2].sequence_order
message	Human-readable explanation.	Sequence order must be unique within the pipeline.

24. Frontend Screen Contract

The generated frontend must contain the following screens or equivalent routed views. Each screen must display the hierarchy breadcrumb CAutomation / CCore / Pipeline Management when inside this module.

Screen	Required capabilities	Required data sources
Hierarchy Browser	Browse Project to Product to Module and select current context.	Project, Product, Module APIs and metadata APIs.
Metadata Administration	Manage and filter type/status/target metadata values where permitted.	Metadata lookup APIs.
Task Catalog	Create, search, update, archive, restore, and inspect reusable Tasks.	Task APIs.
Pipeline List	List pipelines filtered by selected Project/Product/Module context.	Pipeline APIs.
Pipeline Detail	View pipeline fields, status, version, validation result, and	Pipeline, PipelineTask, Task APIs.

	ordered PipelineTasks.	
Pipeline Task Editor	Add reusable Tasks, set sequence_order, required flag, configuration, retry policy, and failure behavior.	Task catalog and PipelineTask APIs.
Execution Detail	Show execution status, task execution statuses, timestamps, messages, and linked reports.	PipelineExecution and PipelineTaskExecution APIs.
Report Viewer	Show planning, validation, execution, task, apply, and verification reports.	ExecutionReport APIs.

25. Exact Generation Readiness Checklist

A generation run is allowed only if the following checklist can be evaluated as passed. A failed item creates a blocking specification gap.

Check ID	Readiness check	Pass condition
GRP-001	Table names are deterministic.	ATS field matrix includes project, product, module, pipeline, task, pipeline_task, pipeline_execution, pipeline_task_execution, and execution_report.
GRP-002	Task reuse is deterministic.	Pipeline and Task remain separate and are connected through pipeline_task.
GRP-003	Context validation is deterministic.	Pipeline creation requires project_id, product_id, and module_id and validates parent integrity.
GRP-004	Status values are deterministic.	SRS approved status values and ATS seed data agree.
GRP-005	API shapes are deterministic.	Request and response DTO names are present in this ATS.
GRP-006	Frontend scope is deterministic.	Required screens are listed with capabilities and data sources.
GRP-007	Reports are deterministic.	Report types and minimum content are listed.
GRP-008	Tests are deterministic.	Testing contract covers database, service, API, frontend, generation, and regression tests.
GRP-009	Repository writes are controlled.	Generation writes to staging first and apply is required before target modification.
GRP-010	Traceability is complete.	Generated artifacts cite SRS requirement IDs or ATS implementation sections.

26. Blocking Non-Generation Conditions

- Do not generate implementation if a required table lacks a primary key, foreign key, or required status rule.
- Do not generate APIs if request and response DTO shapes cannot be mapped to SRS requirements.
- Do not generate frontend screens if required validation envelope fields are missing.
- Do not apply generated files directly to target paths without validation passing.
- Do not create project-level SRS or ATS documents during this module-generation readiness pass.